Sigle : 042.05

Prof. : Madalin Iulian Lupu

TP_1

Réseaux de neurones convolutifs — CNN

Nom : Dekhil Hafedh

https://github.com/Hafdekhil/cat_dog.git

Optimisation d'un classificateur de chiens et de chats (Cats vs Dogs)
CNN à partir de zéro (from scratch) et transfert d'apprentissage (transfer learning)

1. Introduction

La classification d'images par ordinateur constitue un problème fondamental en intelligence artificielle. L'objectif de ce projet était de développer un classificateur d'images capable de distinguer deux classes :

- **Chat** (label 0)
- **Chien** (label 1)

La base de données utilisée provient du jeu de données public **Microsoft Cats vs Dogs**, qui contient des milliers d'images réelles de chats et de chiens prises dans des conditions variées (angles de vue, éclairage, arrière-plan, posture, etc.).
Les objectifs principaux de ce projet étaient :

- D'implémenter un modèle de base (*baseline*),
- D'analyser ses performances,
- D'explorer plusieurs modifications,
- D'identifier la configuration optimale.

Deux approches principales ont été évaluées :

- Un réseau de neurones convolutionnel (**CNN**) entraîné à partir de zéro, sans transfert d'apprentissage,
- Une approche avancée de **transfer learning** utilisant **MobileNetV2**.

Plusieurs essais d'optimisation ont été effectués afin d'obtenir le meilleur taux de précision possible. L'effet de chaque modification a été systématiquement analysé et documenté. Le projet a débuté avec un modèle de base atteignant environ **80,17 %** de précision sur le jeu de test. Après optimisation, le meilleur modèle entraîné à partir de zéro a atteint **89,67 %**, tandis que l'approche de transfert d'apprentissage a permis d'obtenir **96,83 %** sur le jeu de test.

2. Description du modèle de base

2.1 Présentation

Le jeu de données utilisé provient du dataset **Cats vs Dogs**. Les images ont été chargées depuis un fichier ZIP, converties en RGB, redimensionnées à une taille fixe, puis normalisées entre 0 et 1. Le problème traité devient alors une simple classification binaire d'images :

- **0 = Chat**
- **1 = Chien**
-

2.2 Prétraitement initial

La base de données initiale a été limitée à un total de **4000 images**, réparties équitablement comme suit :

- **2000 images de chats**
- **2000 images de chiens**

Les données ont été divisées de manière stratifiée avec random_state = 42 :

- **Entraînement** : 2720 images
- **Validation** : 680 images
- **Test** : 600 images

2.3 Configuration du modèle de base

La configuration initiale du modèle de base utilisait notamment :

- IMG_SIZE = 128
- LEARNING_RATE = 0.0005
- DROPOUT = 0.5
- optimiseur **Adam**
- fonction de perte **binary_crossentropy**

Le pipeline d'entraînement incluait également :

- EarlyStopping
- ReduceLROnPlateau
- ModelCheckpoint

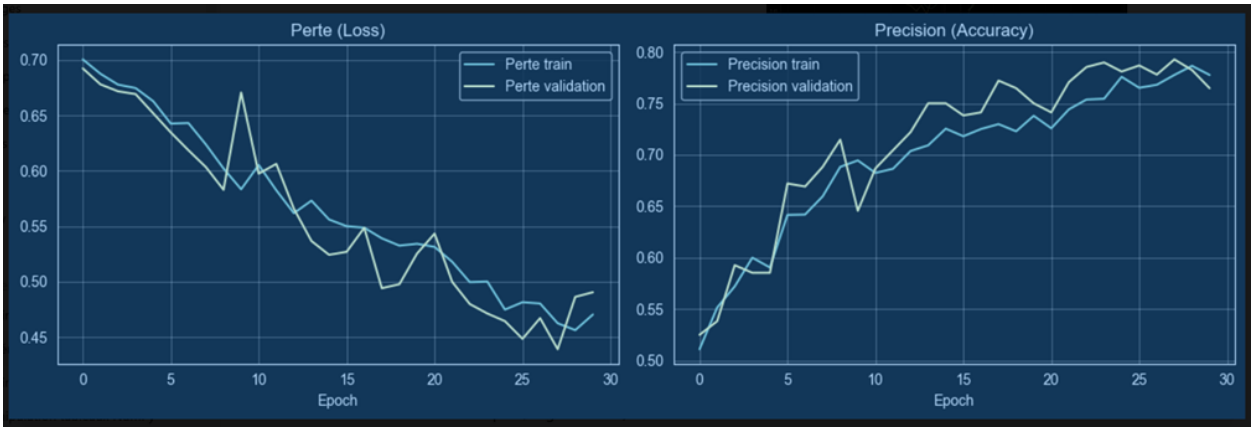
2.4 Résultats initiaux

La validation du modèle de référence avec les conditions mentionnées ci-dessus a donné :

- **Taux de précision: 80,17 %**
- **F1-score : 0,801**

Ce résultat, supérieur au taux de précision minimal mentionné dans le TP, constituait une base solide avec un potentiel d'amélioration significatif pour commencer l'optimisation.

Figure 1 : Courbes du modèle de départ : accuracy et loss



Les courbes du modèle de base indiquent que le réseau apprend correctement, mais sa capacité de généralisation demeure limitée. L'écart observé entre les performances d'apprentissage et de

validation suggère qu'une amélioration de l'architecture, de la régularisation et de la quantité de données pourrait générer des gains significatifs.

3. Expériences réalisées

Plusieurs expériences ont été menées afin d'améliorer les performances du modèle. Avant d'entamer les optimisations, le pipeline a été organisé de manière claire et fiable. Cette structuration a permis d'identifier précisément l'impact de chaque modification et de comparer efficacement les différentes expériences. Bien que cette étape n'ait pas directement augmenté la précision du modèle, elle a considérablement amélioré la clarté, la structure et la reproductibilité du travail.

L'objectif était également d'améliorer la capacité du modèle à gérer la diversité naturelle des images. À cette fin, de légères transformations ont été appliquées aux images d'entraînement afin de permettre au réseau d'apprendre des caractéristiques plus robustes. Cette démarche visait à renforcer la capacité du modèle à reconnaître efficacement des images inédites.

3.1 Architecture plus profonde avec BatchNormalization

Le modèle a été remplacé par une architecture plus profonde comprenant :

- trois blocks convolutionnels,
- deux convolutions par bloc,
- BatchNormalization,
- MaxPooling2D,
- GlobalAveragePooling2D.

Le modèle initial semblait trop simple pour exploiter pleinement la richesse visuelle du jeu de données. L'augmentation de la profondeur du réseau a conduit à une amélioration significative des performances et à une stabilité accrue lors de l'entraînement.

Les premiers résultats ont également révélé des signes de sous-adaptation attribuables à une régularisation excessive. Les tests ont donc été conçus pour identifier le niveau optimal de régularisation, maximisant ainsi les performances tout en limitant le surapprentissage.

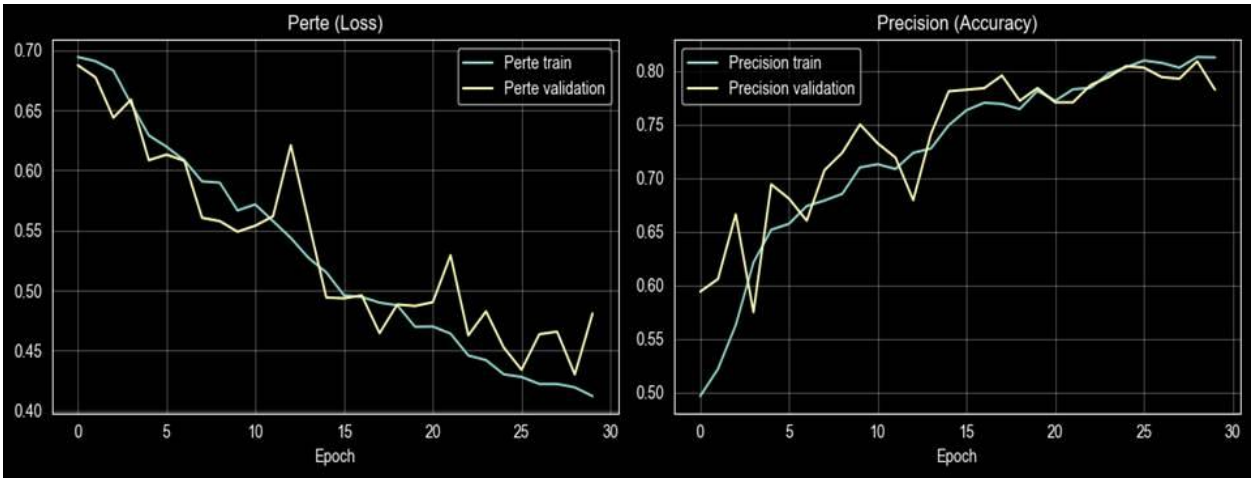
Observation :Le meilleur compromis a été obtenu avec :

- L2_FACTOR = 0.0005
- DROPOUT_RATE = 0.3

3.2 Augmentation du nombre d'images à 3000 par classe

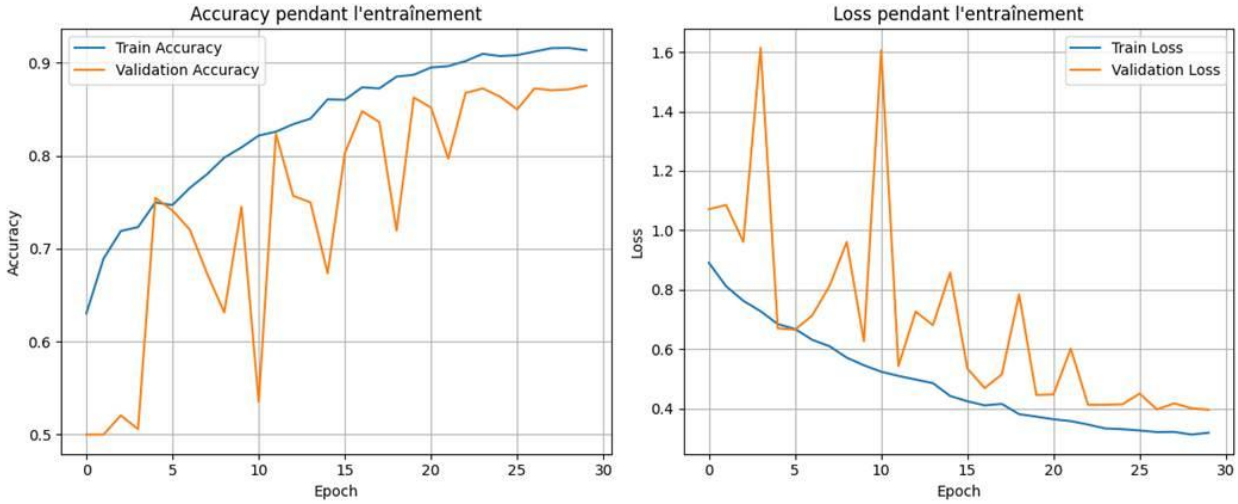
Étant donné que, dans les séries d'expériences précédentes avec le learning rate et le dropout, les meilleurs résultats ne dépassaient pas un taux de précision d'environ 81 %, nous avons décidé d'optimiser un nouveau facteur, tout en conservant L2_FACTOR = 0.0005 et DROPOUT_RATE = 0.3.

Figure 2 : Courbes du meilleur modèle d'apprentissage à partir de zéro avec L2_FACTOR = 0.0005 et DROPOUT_RATE = 0.3.



Dans cette phase expérimentale, la taille de la base de données a été augmentée, portant le nombre maximal d'images par classe de 2000 à 3000. L'hypothèse sous-jacente était que la quantité limitée de données freinait l'amélioration du taux de précision.
Cette modification a eu un impact significatif, permettant d'atteindre un taux de précision supérieur à 85 % sur le jeu de test.

Figure 3 : Courbes du meilleur modèle d'apprentissage à partir de zéro avec L2_FACTOR = 0.0005, DROPOUT_RATE = 0.3 et 3000 images par classe



Résultat :

- **Test accuracy : 86,56 %**
- **F1-score : 0,865**

Avec la configuration suivante :

- MAX_IMAGES_PER_CLASS = 3000
- LEARNING_RATE = 0.0005
- EPOCHS = 30
- BATCH_SIZE = 32
- PATIENCE_EARLYSTOP = 5
- PATIENCE_REDUCE_LR = 2
- REDUCE_FACTOR = 0.5
- L2_FACTOR = 0.0005
- DROPOUT_RATE = 0.3

3.3 Augmentation à 4000 images par classe

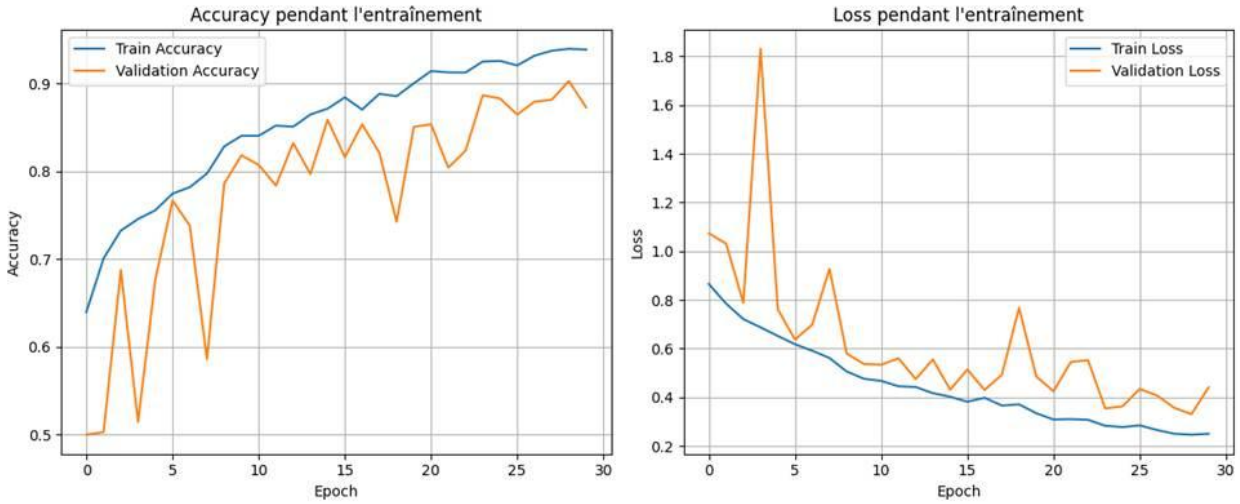
À la suite des résultats obtenus avec 3000 images par classe, il a été jugé pertinent d'évaluer l'effet d'un volume de données encore plus important. Le nombre d'images a ainsi été porté à 4000 par classe.

Ce paramétrage a permis d'obtenir le meilleur résultat pour un apprentissage à partir de zéro, avec

- Taux de précision : 89,67 %
- F1-score : 0,897

Ces résultats confirment que l'augmentation du nombre d'images représente le principal facteur d'amélioration des performances du modèle.

Figure 4 : Courbes du meilleur modèle d'apprentissage à partir de zéro avec L2_FACTOR = 0.0005, DROPOUT_RATE = 0.3 et 4000 images par classe



Résultat final :

- **Test accuracy : 89,67 %**
- **F1-score : 0,897**

Avec la configuration suivante :

- MAX_IMAGES_PER_CLASS = 4000
- IMG_SIZE = 128
- LEARNING_RATE = 0.0005
- EPOCHS = 30
- BATCH_SIZE = 32
- PATIENCE_EARLYSTOP = 5
- PATIENCE_REDUCE_LR = 2
- REDUCE_FACTOR = 0.5
- L2_FACTOR = 0.0005
- DROPOUT_RATE = 0.3

Ces résultats confirment clairement que le levier principal était :

- Augmentation du nombre d'images
- Associée à une régularisation bien réglée (Dropout = 0.3)

L'évolution observée a été la suivante :

- 2000 images/classe → ~81–82 %
- 3000 images/classe → 85–86 %
- 4000 images/classe → 89,67 %

Le meilleur modèle CNN entraîné à partir de zéro a été obtenu en augmentant la base de données à 4000 images par classe sur Google Colab, tout en maintenant une architecture plus profonde intégrant BatchNormalization, GlobalAveragePooling2D, une régularisation L2, un taux de Dropout de 0.3, une légère augmentation des données et des callbacks pour contrôler le surapprentissage.

3.4 Test avec IMG_SIZE = 160

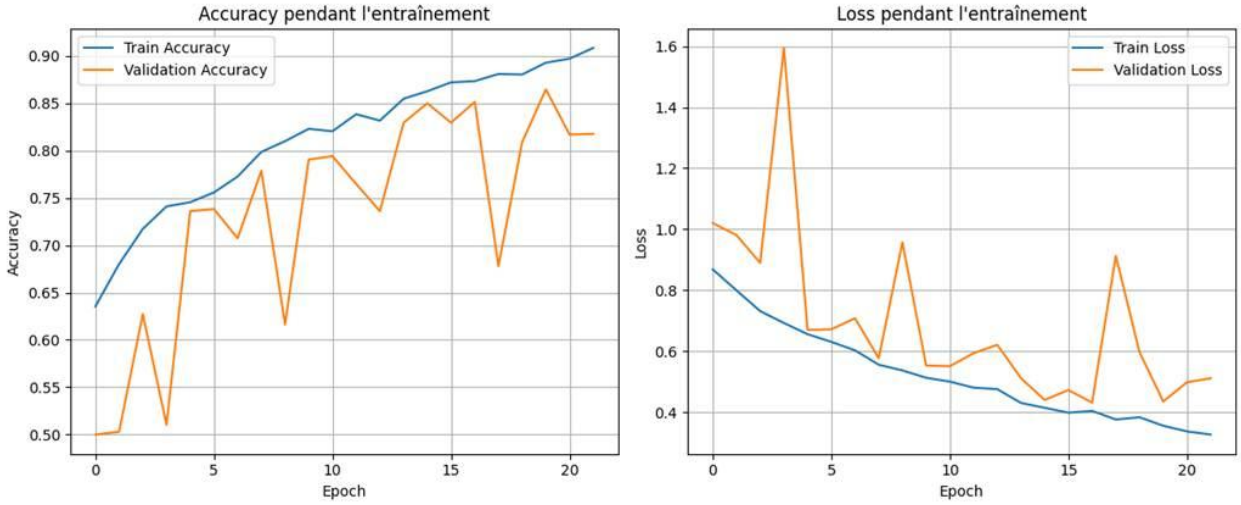
Une dernière tentative a été menée afin d'évaluer l'impact d'une résolution d'image plus élevée, en augmentant la taille des images de 128 à 160 pixels.

Cependant, les résultats obtenus avec IMG_SIZE = 160 se sont révélés inférieurs à ceux obtenus avec IMG_SIZE = 128 :

- Test accuracy : 85,00 %
- F1-score : 0,850

Dans ce projet, la taille d'image de 128 pixels a constitué le meilleur compromis entre richesse de l'information, stabilité des résultats et coût computationnel.

Figure 5 : Courbes du meilleur modèle d'apprentissage à partir de zéro avec L2_FACTOR = 0.0005, DROPOUT_RATE = 0.3, 4000 images et IMG_SIZE = 160



Résultat final

- **Test accuracy : 85%**
- **F1-score : 0.850**

4. Conclusion de la première partie

Au cours de la première partie du projet, plusieurs optimisations ont été réalisées avec succès afin d'améliorer le taux de précision d'un modèle CNN entraîné à partir de zéro.

Tableau 1 : Résumé des principales expériences

Expérience	Modification principale	Accuracy test	F1-score	Observation
Baseline	2000/class, Dropout 0.5	80.17%	0.801	Point de départ
Exp12	Dropout 0.4	81.67%	0.817	Amélioration nette
Exp13	3000/class	85.22%	0.852	Gros gain grâce aux données
Exp14	Dropout 0.3	86.56%	0.865	Meilleure régularisation
Exp16	IMG_SIZE 160	85.00%	0.850	Moins bon que 128
Final	4000/class, 128, Dropout 0.3	89.67%	0.897	Meilleure config

Après plusieurs expériences d'optimisation, le meilleur modèle a été obtenu avec :

- **4000 images par classe**
- IMG_SIZE = 128
- Une architecture CNN plus profonde avec BatchNormalization
- GlobalAveragePooling2D
- une régularisation L2
- un-Dropout à 0.3
- une data augmentation légère
- Des callbacks de contrôle du surapprentissage

Le modèle final a atteint **89,67 % de précision sur le jeu de test,** avec un **F1-score de 0,897**.

La meilleure configuration entraînée à partir de zéro est donc :

- MAX_IMAGES_PER_CLASS = 4000
- IMG_SIZE = 128
- LEARNING_RATE = 0.0005
- EPOCHS = 30
- BATCH_SIZE = 32
- PATIENCE_EARLYSTOP = 5
- PATIENCE_REDUCE_LR = 2
- REDUCE_FACTOR = 0.5
- L2_FACTOR = 0.0005
- DROPOUT_RATE = 0.3

5. Piste avancée : le transfer learning

Le **transfer learning** consiste à réutiliser un modèle pré-entraîné sur des millions d'images, comme **MobileNetV2**, puis à l'adapter à une nouvelle tâche, ici la classification chats/chiens. Cette approche donne généralement des résultats nettement supérieurs.

MobileNetV2 a donc été utilisé afin de comparer cette méthode au meilleur résultat obtenu avec le CNN entraîné à partir de zéro.

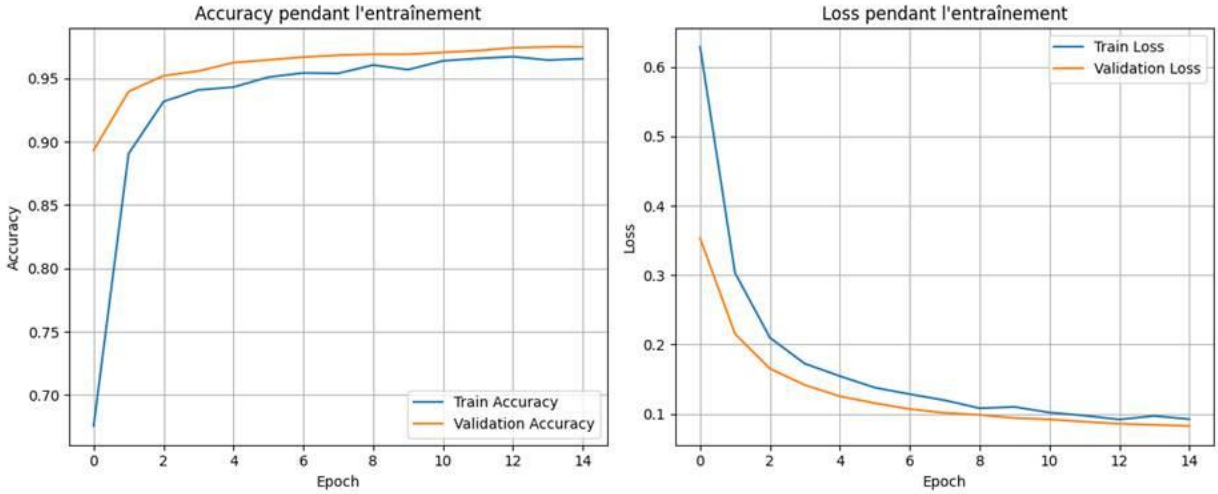
Le modèle **MobileNetV2** a obtenu :

- **Accuracy test : 96,83 %**
- **F1-score : 0,968**
- **Erreurs totales : 38**

Les courbes obtenues, comparées à celles du modèle entraîné à partir de zéro, présentent une grande régularité :

- La précision de validation est très élevée dès les premières epochs,
- Les pertes diminuent de manière régulière,
- Le modèle présente une excellente capacité de généralisation.

Figure 6 : Courbes du modèle MobileNetV2



Résultat transfer learning

- **Test accuracy : 96.83%**
- **F1-score : 0.968**
- **Erreurs totales : 38**

Cette méthode surpasse largement notre modèle entraîné à partir de zéro :

- **CNN from scratch : 89,67 %, F1-score = 0,897**
- **MobileNetV2 : 96,83 %, F1-score = 0,968**

6. Conclusion

Ce projet a permis d'explorer de manière rigoureuse l'optimisation d'un classificateur **Cats vs Dogs** selon deux approches distinctes :

- un **CNN entraîné à partir de zéro**,
- une approche avancée de **transfer learning** avec **MobileNetV2**.

Le modèle de base obtenait environ **80,17 %** sur le jeu de test. Grâce à plusieurs expériences ciblées, le meilleur modèle entraîné à partir de zéro a atteint **89,67 %**, soit une amélioration majeure de près de **10 points**.

Les facteurs qui ont eu le plus d'impact ont été :

- l'augmentation du nombre d'images par classe,
- l'utilisation d'une architecture plus profonde,
- l'ajout de BatchNormalization,
- une régularisation bien dosée,
- le réglage du Dropout à 0.3,
- une data augmentation légère.

Le test avec IMG_SIZE = 160 a montré qu'une résolution d'image plus élevée n'était pas forcément bénéfique dans ce contexte.

Enfin, l'expérience de **transfer learning** avec **MobileNetV2** a atteint un taux de précision de **96,83 %**, confirmant ainsi l'efficacité des modèles pré-entraînés pour des tâches de classification d'images sur des ensembles de données de taille moyenne.

Ce projet démontre donc qu'un **CNN entraîné à partir de zéro**, lorsqu'il est bien optimisé, peut atteindre d'excellents résultats. Cependant, le **transfer learning** permet d'aller encore plus loin et d'obtenir de meilleures performances avec une convergence plus rapide et plus stable.

Tableau 2 : Totale des expériences réaliser sur le desktop

ExpÃ©rience	LR	Accuracy (%)	Loss	F1-score	Erreurs	Temps (s)	LR final	Learning Rate	Epochs	F1-score (ma	Erreurs totale
Exp2_LR_0.0005	0.0005	80.17	0.4332	0.801	119	1172.66	0.000125				
Exp9_Dense(256)_IMG128_LR0.0005_DO0.5	0.0005	79.67	0.4259	0.797	122	793.12	0.000125				
Exp1_LR_0.0003	0.0003	79.5	0.4276	0.795	123	779.4	0.00015				
Exp7_LR0.0005_DO0.5_AUG_30_0.3	0.0005	79.5	0.4393	0.795	123	824.06	0.0005				
Exp6_LR0.0005_Dropout_0.6	0.0005	79.5	0.4198	0.795	123	858.9	0.0005				
Exp8_IMG160_LR0.0005_DO0.5	0.0005	77.67	0.4604	0.777	134	1359.61	0.0005				
Exp4_LR_0.0005_Dropout_0.4	0.0005	77.5	0.4591	0.775	135	751.85	0.0005				
Exp11_BN_GAP_IMG128_LR0.0005_DO0.5	0.0005	77.5	0.4766	0.774	135	1482.3	0.00025				
Exp10_Conv2D(256)_IMG128_LR0.0005_DO0.5	0.0005	77.33	0.4507	0.773	136	856.86	0.000125				
Exp13_MinimalChange_BN_NoLastPool	0.0005	77	0.4693	0.768	138	1539.4	0.00025				
Exp5_LR0.0005_Dropout_0.3	0.0005	76.83	0.454	0.766	139	751.66	0.0005				
Exp3_LR_0.001	0.001	73.67	0.5076	0.736	158	785.06	0.0005				
Exp12_Deux_Conv_BN_GAP_IMG128_LR0.0005_DO0.5	0.0005	61.5	0.6551	0.612	231	1577.96	0.00025				
Exp14_Deep160_Dense256_40epochs	0.0005	50	0.6931	0.333	300	3209.57	4.049999915878288e-06				
Exp2_plus_v1		80.17	0.4789					0.0005	40	0.799	119
Exp3_Scratch_DataAug		79	0.4497			725.41	1.350000002	0.0005	30	0.79	126
Exp3_Scratch_DataAug		79	0.4497			831.22	1.350000002	0.0005	30	0.79	126
Exp4_GlobalAvgPool		71.83	0.5628			706.08	4.500000068	0.0005	30	0.718	169
Exp4_DeeperCNN		75.67	0.5316			1833.4	4.500000068	0.0005	30	0.757	146
Exp5_BatchNormCNN		80.17	0.4211			2786.16	1.350000002	0.0005	30	0.802	119
Exp6_L2_Regularization		80.33	0.5839			2209.06	0.000125	0.0005	24	0.803	118
Exp7_L2_0005_LightAug		80.5	0.5839			2226.11	0.000125	0.0005	24	0.804	117
Exp8_L2_0005_LightAug		77.67	0.5839			1867.23	1.875000089	0.0003	20	0.775	134
Exp9_Scratch_Refined		78.5	0.6035			1867.4	3.125000148	0.0005	20	0.785	129
Exp10_Scratch_Refined		81	0.6054			2315.38	7.812500371	0.0005	25	0.81	114
Exp11_Dropout_04		81.67	0.5622			2329.18	1.562500074	0.0005	25	0.817	110

Le reste des expériences a été réalisé sur **Google Colab**.